# Product Requirements Document

# MediSync MVP

## 1. Product Overview

MediSync is a pharmacy-focused medication availability and inventory management platform for Lebanon.

The MVP helps pharmacies manage medicine stock, batches, expiry dates, sales, invoices, prescriptions, and basic availability publishing. It also provides a public-facing search experience where clients can search for medications and see which connected pharmacies may have them available.

The product starts as a practical pharmacy operations tool and gradually creates a shared medication availability network. The first version should prioritize pharmacy adoption, accurate medicine inventory data, and a reliable foundation for later doctor access, prescription workflows, analytics, and AI-assisted medicine name normalization.

The MVP is intended for:

- Pharmacy staff who need to manage stock, expiry, sales, prescriptions, and invoices.
- Pharmacy owners/managers who need visibility into inventory, revenue, low-stock items, and expiring products.
- Clients/patients who need to find nearby pharmacies that have a medication available.
- Doctors, in a limited early role, who may search medication availability before sending a patient to a pharmacy.
- Platform admins who manage pharmacies, medicine catalog records, users, and data quality.

The project should be built as a secure, maintainable, modular web application suitable for a small student team delivering within a few months for an Amazon mentorship program.

Recommended architecture for this PRD:

- Frontend: Next.js with TypeScript.
- Backend: Django with Django REST Framework.
- Database: PostgreSQL.
- Storage: Private object storage for prescription files, ideally Amazon S3 in production.
- Deployment target: AWS-friendly deployment, such as frontend on Amplify or similar and backend on App Runner or similar.
- Architecture style: Modular monolith, not microservices.

---

# 2. Goals and Non-Goals

## 2.1 MVP Goals

The MVP should accomplish the following:

1. Allow pharmacies to register or be onboarded by an admin.
2. Allow pharmacy staff to log in securely.
3. Support role-based access for platform admins, pharmacy owners, pharmacy staff, doctors, and public users.
4. Allow pharmacies to manage medicine inventory.
5. Track medicine batches, quantities, expiry dates, supplier information, purchase cost, and selling price.
6. Support low-stock and expiring-soon alerts.
7. Allow pharmacies to import inventory from CSV or Excel.
8. Normalize imported medicine names using basic rule-based and fuzzy matching.
9. Allow pharmacies to record sales and generate basic invoices.
10. Allow pharmacies to upload or record prescription information.
11. Publish simplified medicine availability status to a public search page.
12. Let clients search for medications and view pharmacies with available or low-stock status.
13. Provide basic pharmacy dashboard metrics.
14. Provide audit logging for sensitive actions.
15. Provide an admin interface for managing users, pharmacies, and medicine catalog records.

## 2.2 Non-Goals for MVP

The MVP must not attempt to implement:

1. Full national electronic prescription infrastructure.
2. Full doctor-to-pharmacy prescription issuing workflow.
3. Insurance claim processing.
4. Payment processing.
5. Delivery logistics.
6. Clinical diagnosis.
7. Treatment recommendations.
8. AI-generated medical advice.
9. Automatic substitution recommendations without pharmacist validation.
10. Controlled-medication regulatory reporting.
11. Native mobile applications.
12. Multi-country support.
13. Complex POS integrations.
14. Real-time supplier integrations.
15. Advanced fraud detection.
16. Advanced machine learning demand forecasting.
17. Blockchain, event sourcing, Kafka, Kubernetes, or microservices.
18. Public display of exact stock quantities.

19. Public access to prescription data.

---

# 3. Target Users

## 3.1 Platform Admin

A platform admin manages MediSync itself.

Needs:

- Create and manage pharmacy accounts.
- Approve or disable pharmacies.
- Manage medicine catalog data.
- Review import matching issues.
- Manage user roles.
- View audit logs.
- Resolve data quality issues.
- Ensure no unsafe medical content is published.

## 3.2 Pharmacy Owner / Manager

A pharmacy owner or manager controls one pharmacy account.

Needs:

- Manage pharmacy profile and staff users.
- View stock levels and sales overview.
- Add, edit, and archive inventory items.
- Track expiring batches.
- Review low-stock items.
- Record sales and invoices.
- Upload prescriptions.
- Control whether availability is published publicly.
- View activity history for staff actions.

## 3.3 Pharmacy Staff / Pharmacist

A pharmacy staff user performs daily operations.

Needs:

- Search pharmacy inventory quickly.
- Add or update medicine stock.
- Record sales.
- Attach prescriptions to sales when needed.

- View low-stock and expiry alerts.
- Import stock data from spreadsheets if permitted.
- Avoid accidental changes to sensitive records.

## 3.4 Doctor

A doctor has limited MVP access.

Needs:

- Search medication availability.
- View which pharmacies may have a medication available.
- Avoid sending patients to pharmacies without stock.
- No ability to access pharmacy internal inventory, invoices, or prescription files in MVP.

Doctor accounts may be optional in MVP. A doctor can use the public search flow unless doctor-specific login is explicitly implemented.

## 3.5 Client / Patient / Public User

A client searches for medication availability.

Needs:

- Search by medicine name, brand name, generic name, or partial spelling.
- View pharmacies that report available or low-stock status.
- View pharmacy contact details and location.
- Understand that availability should be confirmed with the pharmacy before visiting.
- No login required for MVP.
- No access to internal stock numbers, invoices, or prescriptions.

---

# 4. Core User Stories

## 4.1 Platform Admin

- As a platform admin, I want to create pharmacy accounts, so that pharmacies can join the MediSync pilot.
- As a platform admin, I want to approve or deactivate pharmacies, so that only verified pharmacies appear in public search.
- As a platform admin, I want to manage medicine catalog records, so that search and import matching are consistent.
- As a platform admin, I want to review unmatched imported medicine names, so that the catalog becomes cleaner over time.
- As a platform admin, I want to view audit logs, so that sensitive actions can be traced.

- As a platform admin, I want to manage user roles, so that each user only accesses what they are allowed to access.

## 4.2 Pharmacy Owner / Manager

- As a pharmacy owner, I want to view dashboard metrics, so that I understand stock, sales, and expiry risks.
- As a pharmacy owner, I want to add staff members, so that my team can use the system.
- As a pharmacy owner, I want to import stock from Excel or CSV, so that I can migrate existing inventory quickly.
- As a pharmacy owner, I want to control public availability settings, so that I can choose which inventory appears in search.
- As a pharmacy owner, I want to view low-stock alerts, so that I can reorder medicines before running out.
- As a pharmacy owner, I want to view expiring batches, so that I can reduce waste.
- As a pharmacy owner, I want to view sales and invoices, so that I can track pharmacy performance.

## 4.3 Pharmacy Staff / Pharmacist

- As pharmacy staff, I want to search inventory, so that I can quickly answer customer requests.
- As pharmacy staff, I want to update stock quantities, so that inventory remains accurate.
- As pharmacy staff, I want to add medicine batches with expiry dates, so that the system can warn us before products expire.
- As pharmacy staff, I want to record a sale, so that stock quantities decrease automatically.
- As pharmacy staff, I want to upload a prescription file when needed, so that prescription-linked sales have proper records.
- As pharmacy staff, I want validation warnings, so that I do not accidentally sell more quantity than available.
- As pharmacy staff, I want clear empty and error states, so that I understand what action to take.

## 4.4 Doctor

- As a doctor, I want to search medication availability, so that I can guide patients toward pharmacies that may have the medicine.
- As a doctor, I want to search by brand or generic name, so that I can find relevant alternatives in the catalog.
- As a doctor, I want pharmacy contact details, so that the patient can confirm availability.

## 4.5 Client / Patient

- As a client, I want to search for a medicine, so that I can find nearby pharmacies that may have it.
- As a client, I want to see availability status, so that I know whether a pharmacy is likely to have the medicine.
- As a client, I want to see pharmacy location and contact information, so that I can call or visit.
- As a client, I want the system to handle misspellings, so that I can still find the medicine even if I type the name imperfectly.

- As a client, I want a clear disclaimer, so that I know availability should be confirmed with the pharmacy.

---

# 5. Feature Requirements

## 5.1 Authentication and Role-Based Access

### Description

Users must log in to access protected pharmacy, doctor, and admin features. Public search does not require login.

### User Role Access

- Public user: Can access public search only.
- Doctor: Can access doctor search if doctor login is implemented.
- Pharmacy staff: Can access assigned pharmacy operational pages.
- Pharmacy owner: Can access all pages for their pharmacy, including staff management.
- Platform admin: Can access platform-wide admin pages.

### Required Behavior

- Users log in with email and password.
- Passwords are securely hashed.
- Sessions or tokens must be secure.
- Users may belong to one pharmacy only in MVP, except platform admins.
- Inactive users cannot log in.
- Deactivated pharmacies cannot access pharmacy dashboards.
- Role must be checked on both frontend route guards and backend API permissions.

### Edge Cases

- Wrong password.
- Nonexistent email.
- Inactive user.
- User assigned to inactive pharmacy.
- User attempts to access another pharmacy’s data.
- Expired session.
- Unauthorized API call.

### Acceptance Criteria

- A user can log in with valid credentials.
- Invalid login attempts show a safe generic error.
- Pharmacy staff cannot access admin pages.
- Staff from Pharmacy A cannot access Pharmacy B data.

- Public users cannot access dashboard APIs.
- Deactivated users are blocked.
- Permission tests exist for protected API endpoints.

---

## 5.2 Pharmacy Management

### Description

Platform admins create and manage pharmacies. Pharmacy owners can update limited profile information for their own pharmacy.

### User Role Access

- Platform admin: Full create/read/update/deactivate.
- Pharmacy owner: Read and update own pharmacy profile fields.
- Pharmacy staff: Read own pharmacy profile.
- Public user: Can view public pharmacy information only.

### Required Behavior

Each pharmacy should have:

- Name.
- License or registration number, optional in MVP.
- Address.
- City/area.
- Phone number.
- Optional WhatsApp number.
- Email.
- Latitude and longitude, optional.
- Public visibility status.
- Active/inactive status.
- Created date and updated date.

### Edge Cases

- Duplicate pharmacy name in same area.
- Missing phone number.
- Invalid email.
- Invalid coordinates.
- Pharmacy is inactive but still appears in old availability records.
- Pharmacy disables public visibility.

### Acceptance Criteria

- Admin can create a pharmacy.
- Admin can deactivate a pharmacy.

- Inactive pharmacies do not appear in public search.
- Pharmacy owner can edit allowed profile fields.
- Public search only shows public-safe pharmacy fields.

---

## 5.3 Medicine Catalog

### Description

A canonical catalog of medicines used to normalize pharmacy inventory, imports, sales, and public search.

### User Role Access

- Platform admin: Full access.
- Pharmacy owner/staff: Search and link inventory to catalog records.
- Doctor/public user: Search published catalog records.
- Pharmacy staff cannot create canonical records unless allowed by admin setting.

### Required Behavior

Each medicine catalog record should include:

- Brand name.
- Generic name.
- Strength, for example 500mg.
- Form, for example tablet, syrup, capsule.
- Manufacturer, optional.
- ATC code or classification, optional.
- Search aliases.
- Active/inactive status.
- Notes, non-clinical only.

### Edge Cases

- Same brand with different strengths.
- Same medicine with different forms.
- Misspelled imported names.
- Duplicate catalog records.
- Inactive medicine record used by old inventory records.
- Medicine with missing generic name.

### Acceptance Criteria

- Admin can create, edit, deactivate medicine records.
- Search works by brand name, generic name, and alias.
- Inactive catalog records do not appear in public search.
- Duplicate prevention exists for brand + strength + form where practical.
- Medicine records used in inventory cannot be hard deleted.

## 5.4 Inventory and Batch Management

### Description

Pharmacies can manage stock by medicine and batch. The system tracks quantities, expiry dates, cost, selling price, and availability.

### User Role Access

- Pharmacy owner: Full access for own pharmacy.
- Pharmacy staff: Create/update inventory depending on permissions.
- Platform admin: Read/support access.
- Public user: No direct inventory access.

### Required Behavior

Inventory should be organized around batches.

Each inventory batch should include:

- Pharmacy.
- Medicine catalog record.
- Batch number, optional but recommended.
- Current quantity.
- Initial quantity.
- Expiry date.
- Supplier name, optional.
- Purchase cost per unit, optional.
- Selling price per unit.
- Public availability enabled boolean.
- Low-stock threshold.
- Created by.
- Updated by.

Stock changes should create stock movement records.

Stock movement types:

- Import.
- Manual adjustment.
- Sale.
- Return.
- Correction.
- Expired.
- Removed.

### Edge Cases

- Quantity cannot be negative.
- Selling more than available.
- Expired batch still has stock.
- Duplicate batch number for same medicine.
- Missing expiry date.
- Stock update conflict from two users.
- Medicine catalog record deactivated after inventory exists.
- Public availability disabled for a medicine.

### Acceptance Criteria

- Staff can add a medicine batch.
- Staff can edit quantity through controlled stock adjustment, not silent overwrite.
- System records stock movement for each quantity change.
- Low-stock items are flagged.
- Expiring items are flagged.
- Expired items are not shown as publicly available.
- Public search does not expose exact quantity.
- Inventory list can be filtered by medicine, low stock, expiry, and availability status.

---

## 5.5 CSV / Excel Inventory Import

### Description

Pharmacies can upload inventory spreadsheets to create or update inventory records.

### User Role Access

- Pharmacy owner: Full import access.
- Pharmacy staff: Import access only if permitted.
- Platform admin: Can view import history and resolve unmatched names.

### Required Behavior

Supported file types:

- CSV.
- XLSX.

Required import columns:

- Medicine name.
- Quantity.

Optional import columns:

- Generic name.
- Strength.
- Form.
- Batch number.
- Expiry date.
- Supplier.
- Purchase cost.
- Selling price.
- Low-stock threshold.

Import flow:

1. User uploads file.
2. System validates file format.
3. System parses rows.
4. System attempts medicine matching.
5. System displays preview.
6. User confirms matched rows.
7. User manually resolves unmatched rows or skips them.
8. System creates inventory batches and stock movements.
9. System stores import summary.

Medicine matching should use:

- Exact normalized match.
- Case-insensitive match.
- Alias match.
- Fuzzy string match.
- Manual confirmation for uncertain matches.

## Edge Cases

- Empty file.
- Unsupported file type.
- Missing required columns.
- Invalid quantity.
- Invalid expiry date.
- Duplicate rows.
- Same medicine appears multiple times.
- Fuzzy match confidence too low.
- File too large.
- Import partially fails.
- User cancels before confirmation.

### Acceptance Criteria

- User can upload a valid CSV or XLSX file.
- System displays import preview before writing to database.
- Invalid rows are shown with clear errors.
- System does not create inventory records until user confirms.
- Import creates stock movement records.
- Unmatched medicines are not silently invented.
- Import summary shows created, skipped, failed, and unmatched row counts.

---

## 5.6 Public Medication Availability Search

### Description

Public users, clients, and doctors can search for a medication and view connected pharmacies that report availability.

### User Role Access

- Public user: Search public availability.
- Doctor: Same as public in MVP, unless doctor dashboard is implemented.
- Pharmacy users: Can preview how their pharmacy appears publicly.
- Admin: Can search and debug availability.

### Required Behavior

Search should support:

- Brand name.
- Generic name.
- Alias.
- Partial name.
- Basic misspelling tolerance.
- Filters by city/area.
- Optional filter by availability status.

Availability statuses:

- Available.
- Low stock.
- Unavailable.
- Unknown.

Public result should show:

- Medicine name.
- Pharmacy name.

- Area/city.
- Phone number.
- Optional WhatsApp/contact link.
- Availability status.
- Last updated timestamp.
- Disclaimer to confirm with pharmacy.

Public result must not show:

- Exact stock quantity.
- Purchase cost.
- Supplier.
- Prescription records.
- Internal notes.
- Staff names.
- Sales data.

### Edge Cases

- No matching medicine.
- Matching medicine but no pharmacy has it.
- Pharmacy disabled public visibility.
- Inventory is stale.
- Medicine is expired.
- Medicine exists in multiple strengths/forms.
- User misspells medicine name.
- Public search gets too many results.

### Acceptance Criteria

- Public user can search without login.
- Search results only show public-safe data.
- Expired stock does not count as available.
- Deactivated pharmacies do not appear.
- Results show last updated time.
- Results include confirmation disclaimer.
- Exact quantity is never exposed publicly.

---

## 5.7 Sales and Invoice Management

### Description

Pharmacy staff can record basic sales and generate invoice records. Sales reduce inventory quantities.

### User Role Access

- Pharmacy owner: Full access.

- Pharmacy staff: Create sales and view recent sales.
- Platform admin: Support/read access if needed.
- Public user: No access.

### Required Behavior

A sale should include:

- Pharmacy.
- Sale date/time.
- Staff user.
- Line items.
- Medicine.
- Quantity.
- Unit price.
- Discount, optional.
- Total amount.
- Payment method, optional.
- Prescription required boolean.
- Prescription attachment/reference, optional.
- Notes, optional.

When a sale is recorded:

- The system checks available stock.
- The system deducts quantity from appropriate batch.
- The system creates stock movement records.
- The system creates an invoice record.
- The system links prescription if uploaded.

Batch deduction strategy for MVP:

- Use earliest expiry first, FEFO.
- If no expiry date exists, use oldest created batch.

### Edge Cases

- Insufficient stock.
- Multiple batches needed to fulfill one sale.
- Expired batch selected.
- Sale cancelled after creation.
- Mistyped quantity.
- Missing selling price.
- Prescription required but no prescription attached.
- User records sale while another user changes stock.

### Acceptance Criteria

- Staff can create a sale with one or more line items.

- Sale total is calculated correctly.
- Stock decreases after sale.
- Stock movement records are created.
- Sale is blocked if stock is insufficient.
- Invoice can be viewed after sale.
- Prescription-required medicine can trigger warning when no prescription is attached.
- Invoice records cannot be hard deleted by staff.

---

## 5.8 Prescription Record Management

### Description

Pharmacy users can attach prescription records to sales or store basic prescription information.

### User Role Access

- Pharmacy owner: Full access for own pharmacy.
- Pharmacy staff: Create and view prescription records for own pharmacy.
- Platform admin: Restricted support access only if explicitly allowed.
- Doctor: No access to pharmacy prescription records in MVP.
- Public user: No access.

### Required Behavior

Prescription record may include:

- Pharmacy.
- Linked sale, optional.
- Patient name, optional but sensitive.
- Patient phone, optional but sensitive.
- Doctor name, optional.
- Prescription date, optional.
- Uploaded file, optional.
- Notes, optional.
- Created by.
- Created at.
- Updated at.

Prescription files must be private.

Rules:

- Prescription files must never use public URLs.
- Backend must authorize file access before serving.
- Prescription data must be excluded from public APIs.
- Prescription views/downloads should be audit logged.

- Avoid storing more patient data than necessary.

### Edge Cases

- Upload unsupported file type.
- Upload too large.
- User attempts to access another pharmacy’s prescription.
- Prescription attached to wrong sale.
- Prescription deleted accidentally.
- Missing patient details.
- Duplicate upload.

### Acceptance Criteria

- Staff can upload a prescription file.
- Prescription file is stored privately.
- Only authorized users can view/download it.
- Public users cannot access prescription data or files.
- Prescription access is audit logged.
- File upload has size/type validation.

---

## 5.9 Dashboard and Alerts

### Description

Pharmacy users see a dashboard with operational metrics and alert lists.

### User Role Access

- Pharmacy owner: Full dashboard.
- Pharmacy staff: Operational dashboard.
- Platform admin: Admin analytics separately.
- Public user: No access.

### Required Behavior

Dashboard should show:

- Total medicines in inventory.
- Low-stock count.
- Expiring-soon count.
- Expired-stock count.
- Sales today.
- Revenue today.
- Recently updated inventory.
- Recent sales.
- Import status, if any.

- Availability publishing status.

Alert types:

- Low stock.
- Expiring soon.
- Expired batch.
- Unmatched imported medicines.
- Missing price.
- Missing expiry date.

### Edge Cases

- New pharmacy has no inventory.
- No sales yet.
- Import in progress.
- Failed import.
- Large inventory causing slow dashboard.
- User lacks permission for revenue metrics.

### Acceptance Criteria

- Dashboard loads for authorized pharmacy users.
- Empty state appears for new pharmacy.
- Low-stock and expiry alerts are accurate.
- Staff without financial permission cannot view revenue if permission setting exists.
- Dashboard queries are scoped to the user’s pharmacy.

---

## 5.10 Audit Logging

### Description

The system records sensitive and important actions for traceability.

### User Role Access

- Platform admin: Can view platform audit logs.
- Pharmacy owner: Can view own pharmacy audit logs.
- Pharmacy staff: No audit log view in MVP unless needed.

### Required Behavior

Audit logs should record:

- User.
- Pharmacy, if applicable.
- Action type.

- Entity type.
- Entity ID.
- Timestamp.
- IP address, if available.
- Summary.
- Before/after diff for critical changes, where practical.

Actions to log:

- Login success/failure, optional.
- User role changes.
- Pharmacy activation/deactivation.
- Inventory quantity changes.
- Batch creation/edit.
- Sale creation/cancellation.
- Prescription upload/view/download.
- Import confirmation.
- Medicine catalog changes.

### Edge Cases

- Action fails halfway.
- User deleted or deactivated later.
- Entity deleted later.
- Missing IP address.
- Bulk import creates many logs.

### Acceptance Criteria

- Sensitive actions create audit records.
- Audit records are immutable from normal app flows.
- Pharmacy owner only sees own pharmacy logs.
- Platform admin can filter audit logs by pharmacy, user, action, and date.

---

# 6. Pages and User Flows

# 6.1 Public Landing Page

### Purpose

Explain MediSync briefly and provide entry to medication search.

### Main Components

- Header.
- Logo/name.

- Search entry point.
- Brief explanation.
- Pharmacy login link.
- Disclaimer.

### User Actions

- Start medication search.
- Navigate to login.
- Learn what MediSync does.

### Navigation Behavior

- Search redirects to public search results.
- Login redirects to login page.

### Empty/Loading/Error States

- Not applicable beyond normal loading.

---

## 6.2 Public Medication Search Page

### Purpose

Allow clients/doctors to search medicine availability.

### Main Components

- Search input.
- Area/city filter.
- Medicine strength/form selector if multiple matches exist.
- Results list.
- Pharmacy contact cards.
- Availability badges.
- Last updated timestamp.
- Disclaimer.

### User Actions

- Search medicine.
- Filter by area.
- Select medicine variant.
- Call pharmacy using displayed contact.
- Clear search.

### Navigation Behavior

- Search updates results.
- Result cards may link to pharmacy detail page if implemented.
- No login required.

### Empty/Loading/Error States

- Empty: “Search for a medicine to see availability.”
- No results: “No connected pharmacies currently show this medicine as available.”
- Loading: skeleton result cards.
- Error: “Search failed. Please try again.”

---

## 6.3 Login Page

### Purpose

Allow staff, owners, admins, and optional doctors to log in.

### Main Components

- Email field.
- Password field.
- Submit button.
- Forgot password placeholder, optional.
- Error message area.

### User Actions

- Enter credentials.
- Submit login form.

### Navigation Behavior

After login:

- Platform admin goes to admin dashboard.
- Pharmacy owner/staff goes to pharmacy dashboard.
- Doctor goes to doctor search page, if implemented.

### Empty/Loading/Error States

- Loading: disable submit button.
- Error: generic invalid login message.
- Unauthorized: show account inactive or contact admin message where appropriate.

---

## 6.4 Pharmacy Dashboard

### Purpose

Show pharmacy operational overview.

### Main Components

- Metrics cards.
- Low-stock alerts.
- Expiring-soon alerts.
- Recent sales.
- Recent inventory changes.
- Quick actions.

### User Actions

- Go to inventory.
- Add stock.
- Record sale.
- Start import.
- View alerts.

### Navigation Behavior

- Sidebar or top nav links to inventory, sales, prescriptions, imports, settings.
- Alert click opens filtered inventory list.

### Empty/Loading/Error States

- Empty: “No inventory yet. Import a spreadsheet or add your first medicine.”
- Loading: metrics skeleton.
- Error: retry button.

---

## 6.5 Inventory List Page

### Purpose

Manage all pharmacy inventory items and batches.

### Main Components

- Search input.
- Filters: low stock, expiring soon, expired, public availability.
- Inventory table.
- Add batch button.

- Import button.
- Pagination.

### User Actions

- Search inventory.
- Filter inventory.
- Open batch detail.
- Add batch.
- Adjust stock.
- Toggle public availability where allowed.

### Navigation Behavior

- Clicking row opens batch detail/edit page.
- Add button opens add batch form.
- Import button opens import flow.

### Empty/Loading/Error States

- Empty: “No inventory records found.”
- Loading: table skeleton.
- Error: retry message.
- Filter empty: “No items match this filter.”

---

## 6.6 Add/Edit Inventory Batch Page

### Purpose

Create or update a medicine batch.

### Main Components

- Medicine search/select.
- Batch number.
- Quantity.
- Expiry date.
- Supplier.
- Purchase cost.
- Selling price.
- Low-stock threshold.
- Public availability toggle.
- Save button.

### User Actions

- Select medicine.

- Enter batch details.
- Save.
- Cancel.

### Navigation Behavior

- Save returns to inventory detail or list.
- Cancel returns to previous page.

### Empty/Loading/Error States

- Medicine search loading.
- Validation errors beside fields.
- Save error message.

---

## 6.7 Inventory Batch Detail Page

### Purpose

View batch details and stock movement history.

### Main Components

- Batch summary.
- Current quantity.
- Expiry status.
- Price information.
- Availability status.
- Stock movement timeline.
- Adjustment button.

### User Actions

- Adjust stock.
- Edit metadata.
- View movement history.
- Disable public availability.

### Navigation Behavior

- Back to inventory list.
- Edit opens edit form.
- Adjust opens adjustment modal.

### Empty/Loading/Error States

- No movements: “No movements recorded yet.”

- Loading: detail skeleton.
- Error: not found or unauthorized.

---

## 6.8 CSV / Excel Import Page

### Purpose

Upload and confirm stock import.

### Main Components

- File upload area.
- Required column instructions.
- Upload button.
- Import preview table.
- Match confidence indicators.
- Row error messages.
- Confirm import button.
- Cancel button.

### User Actions

- Upload file.
- Review parsed rows.
- Resolve unmatched medicines.
- Confirm import.
- Cancel import.

### Navigation Behavior

- Successful import redirects to import summary.
- Cancel returns to inventory list.

### Empty/Loading/Error States

- Empty: upload instructions.
- Loading: parsing/importing spinner.
- Error: invalid file or column error.
- Partial errors: row-level validation messages.

---

## 6.9 Import Summary Page

### Purpose

Show result of completed import.

### Main Components

- Created count.
- Skipped count.
- Failed count.
- Unmatched count.
- Link to inventory.
- Download error report, optional.

### User Actions

- View created inventory.
- Retry failed rows, optional.
- Return to dashboard.

### Navigation Behavior

- Inventory link opens filtered inventory list.

### Empty/Loading/Error States

- Error if import record not found.

---

## 6.10 Sales / Invoices List Page

### Purpose

View sales and invoices.

### Main Components

- Date filter.
- Search by invoice number.
- Sales table.
- Create sale button.
- Total revenue summary.

### User Actions

- Create sale.
- View invoice.
- Filter by date.
- Search invoice.

### Navigation Behavior

- Clicking invoice opens invoice detail.

- Create sale opens sale form.

### Empty/Loading/Error States

- Empty: “No sales recorded yet.”
- Loading: table skeleton.
- Error: retry.

---

## 6.11 Create Sale Page

### Purpose

Record a pharmacy sale and generate invoice.

### Main Components

- Medicine line item selector.
- Quantity field.
- Unit price.
- Discount.
- Prescription required warning.
- Prescription upload field, optional.
- Total calculation.
- Submit button.

### User Actions

- Add medicine line item.
- Remove line item.
- Upload prescription.
- Confirm sale.

### Navigation Behavior

- Successful sale opens invoice detail.
- Cancel returns to sales list.

### Empty/Loading/Error States

- Medicine search loading.
- Insufficient stock error.
- Missing price error.
- Upload error.
- Submit error.

---

## 6.12 Invoice Detail Page

### Purpose

View generated invoice.

### Main Components

- Invoice number.
- Date/time.
- Staff user.
- Pharmacy info.
- Line items.
- Totals.
- Prescription link if authorized.
- Print button, optional.

### User Actions

- Print invoice.
- View linked prescription if authorized.
- Return to sales list.

### Navigation Behavior

- Back button returns to sales list.

### Empty/Loading/Error States

- Not found.
- Unauthorized.
- Loading skeleton.

---

## 6.13 Prescription Records Page

### Purpose

View and manage prescription records for a pharmacy.

### Main Components

- Search/filter list.
- Patient name field, if stored.
- Doctor name.
- Prescription date.
- Linked sale.

- Upload button.
- Access log indicator, optional.

### User Actions

- Upload prescription.
- View prescription.
- Link to sale.
- Filter records.

### Navigation Behavior

- Record click opens prescription detail.
- Upload opens upload form/modal.

### Empty/Loading/Error States

- Empty: “No prescriptions stored yet.”
- Loading.
- Unauthorized.
- File preview error.

---

## 6.14 Pharmacy Settings Page

### Purpose

Manage pharmacy profile and availability publishing settings.

### Main Components

- Pharmacy details form.
- Public visibility toggle.
- Contact info.
- Location fields.
- Staff management link.
- Save button.

### User Actions

- Update contact information.
- Toggle public visibility.
- Save settings.

### Navigation Behavior

- Save stays on page with success message.
- Staff link opens staff management.

### Empty/Loading/Error States

- Validation errors.
- Save failed error.
- Loading state.

---

## 6.15 Staff Management Page

### Purpose

Allow pharmacy owner to manage staff users.

### Main Components

- Staff list.
- Invite/create user form.
- Role selector.
- Active/inactive toggle.

### User Actions

- Add staff.
- Change role.
- Deactivate staff.
- Reset password flow, optional.

### Navigation Behavior

- Create user stays on page and refreshes list.

### Empty/Loading/Error States

- Empty: “No staff users added yet.”
- Error for duplicate email.
- Unauthorized for non-owner.

---

## 6.16 Admin Dashboard

### Purpose

Platform-level management for MediSync admins.

### Main Components

- Pharmacy count.

- Active users.
- Medicine catalog count.
- Recent imports.
- Recent audit logs.
- Quick links.

### User Actions

- Manage pharmacies.
- Manage catalog.
- View audit logs.
- Review unmatched imports.

### Navigation Behavior

- Admin sidebar links to admin sections.

### Empty/Loading/Error States

- Loading.
- Error.
- Empty metrics for new system.

---

## 6.17 Admin Pharmacy Management Page

### Purpose

Create and manage pharmacy accounts.

### Main Components

- Pharmacy table.
- Create pharmacy button.
- Active/inactive status.
- Public visibility status.
- Search/filter.

### User Actions

- Create pharmacy.
- Edit pharmacy.
- Deactivate pharmacy.
- View pharmacy details.

### Navigation Behavior

- Row opens pharmacy detail page.

- Create opens form.

### Empty/Loading/Error States

- Empty: “No pharmacies created yet.”
- Error state.
- Loading state.

---

## 6.18 Admin Medicine Catalog Page

### Purpose

Manage canonical medicine records.

### Main Components

- Medicine table.
- Search.
- Add medicine button.
- Duplicate warning.
- Alias editor.

### User Actions

- Create medicine.
- Edit medicine.
- Deactivate medicine.
- Add aliases.
- Resolve unmatched names.

### Navigation Behavior

- Row opens medicine edit page.

### Empty/Loading/Error States

- Empty catalog state.
- Search empty state.
- Duplicate validation error.

---

## 6.19 Admin Audit Logs Page

### Purpose

View sensitive system actions.

### Main Components

- Filter by pharmacy.
- Filter by user.
- Filter by action.
- Date range.
- Log table.
- Detail drawer.

### User Actions

- Filter logs.
- Open log detail.

### Navigation Behavior

- Stays on page with query filters.

### Empty/Loading/Error States

- Empty logs.
- Loading.
- Error.

---

# 7. Data Model

Use PostgreSQL as the primary database.

Names below are conceptual. Claude Code may map them to Django models using snake_case table names.

## 7.1 User

Fields:

- id: UUID primary key.
- email: unique, required.
- password_hash: handled by Django auth.
- first_name: string.
- last_name: string.
- role: enum.
- PLATFORM_ADMIN.
- PHARMACY_OWNER.
- PHARMACY_STAFF.
- DOCTOR.
- pharmacy_id: FK to Pharmacy, nullable for platform admin.

- is_active: boolean.
- created_at: timestamp.
- updated_at: timestamp.
- last_login_at: timestamp, optional.

Validation:

- Email must be unique.
- Pharmacy users must have pharmacy_id.
- Platform admins should not require pharmacy_id.
- Public users do not need database accounts in MVP.

Relationships:

- User belongs to Pharmacy, optional.
- User creates StockMovements.
- User creates Sales.
- User creates AuditLogs.

---

## 7.2 Pharmacy

Fields:

- id: UUID primary key.
- name: string, required.
- license_number: string, optional.
- address: text.
- city: string.
- area: string.
- phone: string.
- whatsapp: string, optional.
- email: string, optional.
- latitude: decimal, optional.
- longitude: decimal, optional.
- is_active: boolean.
- is_public: boolean.
- created_at: timestamp.
- updated_at: timestamp.

Validation:

- Name required.
- Phone required for public pharmacies.
- Latitude/longitude must be valid if provided.
- Inactive pharmacies must not appear in public search.

Relationships:

- Pharmacy has many Users.
- Pharmacy has many InventoryBatches.
- Pharmacy has many Sales.
- Pharmacy has many PrescriptionRecords.
- Pharmacy has many AuditLogs.

---

## 7.3 Medicine

Fields:

- id: UUID primary key.
- brand_name: string, required.
- generic_name: string, optional.
- strength: string, optional.
- form: string, optional.
- manufacturer: string, optional.
- classification: string, optional.
- notes: text, optional.
- is_active: boolean.
- created_at: timestamp.
- updated_at: timestamp.

Validation:

- Brand name required.
- Prevent exact duplicate active records where brand_name + strength + form match.
- Do not hard delete medicines used by inventory records.

Relationships:

- Medicine has many MedicineAliases.
- Medicine has many InventoryBatches.
- Medicine has many SaleItems.
- Medicine has many ImportRows.

---

## 7.4 MedicineAlias

Fields:

- id: UUID primary key.
- medicine_id: FK to Medicine.
- alias: string, required.

- alias_type: enum.
- BRAND_VARIATION.
- GENERIC.
- MISSPELLING.
- IMPORT_NAME.
- OTHER.
- created_at: timestamp.

Validation:

- Alias cannot be empty.
- Alias should be normalized for search.
- Duplicate alias for same medicine should be prevented.

Relationships:

- Alias belongs to Medicine.

---

## 7.5 InventoryBatch

Fields:

- id: UUID primary key.
- pharmacy_id: FK to Pharmacy.
- medicine_id: FK to Medicine.
- batch_number: string, optional.
- initial_quantity: integer.
- current_quantity: integer.
- expiry_date: date, optional.
- supplier_name: string, optional.
- purchase_cost: decimal, optional.
- selling_price: decimal, required.
- low_stock_threshold: integer, default 5.
- public_availability_enabled: boolean, default true.
- is_archived: boolean, default false.
- created_by_id: FK to User.
- updated_by_id: FK to User.
- created_at: timestamp.
- updated_at: timestamp.

Validation:

- current_quantity >= 0.
- initial_quantity >= 0.
- low_stock_threshold >= 0.
- selling_price >= 0.

- purchase_cost >= 0 if provided.
- Expired batches should not count as public available.
- Archived batches should not count as available.

Relationships:

- InventoryBatch belongs to Pharmacy.
- InventoryBatch belongs to Medicine.
- InventoryBatch has many StockMovements.

---

## 7.6 StockMovement

Fields:

- id: UUID primary key.
- pharmacy_id: FK to Pharmacy.
- inventory_batch_id: FK to InventoryBatch.
- medicine_id: FK to Medicine.
- movement_type: enum.
- IMPORT.
- MANUAL_ADJUSTMENT.
- SALE.
- RETURN.
- CORRECTION.
- EXPIRED.
- REMOVED.
- quantity_delta: integer.
- quantity_before: integer.
- quantity_after: integer.
- reason: text, optional.
- sale_id: FK to Sale, nullable.
- created_by_id: FK to User.
- created_at: timestamp.

Validation:

- quantity_after cannot be negative.
- movement must be immutable after creation except by admin-level correction process.
- movement pharmacy must match batch pharmacy.

Relationships:

- StockMovement belongs to InventoryBatch.
- StockMovement may belong to Sale.

---

## 7.7 Sale

Fields:

- id: UUID primary key.
- invoice_number: string, unique.
- pharmacy_id: FK to Pharmacy.
- staff_user_id: FK to User.
- sale_datetime: timestamp.
- subtotal: decimal.
- discount_total: decimal, default 0.
- total: decimal.
- payment_method: enum, optional.
- CASH.
- CARD.
- OTHER.
- status: enum.
- COMPLETED.
- CANCELLED.
- prescription_record_id: FK to PrescriptionRecord, nullable.
- notes: text, optional.
- created_at: timestamp.
- updated_at: timestamp.

Validation:

- total >= 0.
- Sale must have at least one SaleItem.
- Completed sale cannot be edited directly.
- Cancellation should create reversal stock movement if implemented.

Relationships:

- Sale belongs to Pharmacy.
- Sale has many SaleItems.
- Sale may have one PrescriptionRecord.

---

## 7.8 SaleItem

Fields:

- id: UUID primary key.
- sale_id: FK to Sale.
- medicine_id: FK to Medicine.
- inventory_batch_id: FK to InventoryBatch, nullable if split across batches requires SaleItemBatchAllocation.

- quantity: integer.
- unit_price: decimal.
- discount: decimal, default 0.
- line_total: decimal.

Validation:

- quantity > 0.
- unit_price >= 0.
- line_total calculated from quantity, price, discount.
- Medicine must belong to available inventory for that pharmacy.

Relationships:

- SaleItem belongs to Sale.
- SaleItem references Medicine.
- SaleItem may reference InventoryBatch.

Optional improvement:

Use SaleItemBatchAllocation if a sale line consumes multiple batches.

---

## 7.9 PrescriptionRecord

Fields:

- id: UUID primary key.
- pharmacy_id: FK to Pharmacy.
- sale_id: FK to Sale, nullable.
- patient_name: string, optional.
- patient_phone: string, optional.
- doctor_name: string, optional.
- prescription_date: date, optional.
- file_path: string, optional.
- file_original_name: string, optional.
- file_mime_type: string, optional.
- file_size: integer, optional.
- notes: text, optional.
- created_by_id: FK to User.
- created_at: timestamp.
- updated_at: timestamp.

Validation:

- File type must be allowed.
- File size must be below configured max.
- Prescription must belong to same pharmacy as linked sale.

- Prescription files must not be public.

Relationships:

- PrescriptionRecord belongs to Pharmacy.
- PrescriptionRecord may belong to Sale.
- PrescriptionRecord created by User.

---

## 7.10 InventoryImport

Fields:

- id: UUID primary key.
- pharmacy_id: FK to Pharmacy.
- uploaded_by_id: FK to User.
- original_filename: string.
- status: enum.
- UPLOADED.
- PARSED.
- CONFIRMED.
- FAILED.
- CANCELLED.
- total_rows: integer.
- valid_rows: integer.
- invalid_rows: integer.
- matched_rows: integer.
- unmatched_rows: integer.
- created_count: integer.
- skipped_count: integer.
- error_summary: text, optional.
- created_at: timestamp.
- confirmed_at: timestamp, optional.

Validation:

- Only confirmed imports write inventory records.
- Import belongs to one pharmacy.

Relationships:

- InventoryImport has many InventoryImportRows.

---

## 7.11 InventoryImportRow

Fields:

- id: UUID primary key.
- import_id: FK to InventoryImport.
- row_number: integer.
- raw_medicine_name: string.
- normalized_name: string.
- matched_medicine_id: FK to Medicine, nullable.
- match_confidence: decimal, optional.
- quantity: integer, nullable.
- batch_number: string, optional.
- expiry_date: date, optional.
- supplier_name: string, optional.
- purchase_cost: decimal, optional.
- selling_price: decimal, optional.
- status: enum.
- VALID_MATCHED.
- VALID_UNMATCHED.
- INVALID.
- SKIPPED.
- IMPORTED.
- error_message: text, optional.
- raw_data: JSONB.

Validation:

- Row status must reflect validation result.
- Invalid rows should not be imported.
- Unmatched rows require manual medicine selection or skip.

---

## 7.12 AuditLog

Fields:

- id: UUID primary key.
- actor_user_id: FK to User, nullable.
- pharmacy_id: FK to Pharmacy, nullable.
- action: string.
- entity_type: string.
- entity_id: UUID or string.
- summary: text.
- before_data: JSONB, optional.
- after_data: JSONB, optional.
- ip_address: string, optional.

- user_agent: text, optional.
- created_at: timestamp.

Validation:

- Audit logs should not be editable from normal application flows.
- Sensitive data should be minimized in before/after JSON.

---

# 8. Authentication and Authorization

## 8.1 Authentication Requirements

- Use Django authentication as the backend identity foundation.
- For API authentication, use secure session authentication or JWT depending on frontend/backend deployment.
- Password reset can be stubbed or implemented later if timeline is tight.
- Public search must not require authentication.
- Admin area must require platform admin role.
- Pharmacy dashboard must require pharmacy user role and active pharmacy.
- Doctor dashboard, if implemented, must require doctor role.

## 8.2 Roles

Roles:

1. PLATFORM_ADMIN
2. PHARMACY_OWNER
3. PHARMACY_STAFF
4. DOCTOR

Public users have no role and no login in MVP.

## 8.3 Permission Rules

Platform admin:

- Can manage all pharmacies.
- Can manage all users.
- Can manage medicine catalog.
- Can view audit logs.
- Can view import issues.

Pharmacy owner:

- Can manage own pharmacy profile.
- Can manage own pharmacy staff.

- Can manage inventory.
- Can create/import stock.
- Can record sales.
- Can view own pharmacy reports.
- Can view own pharmacy audit logs.
- Can manage public availability settings.

Pharmacy staff:

- Can view own pharmacy dashboard.
- Can manage inventory if permission enabled.
- Can record sales.
- Can upload prescriptions.
- Cannot manage staff.
- Cannot access platform admin pages.
- Cannot access other pharmacies.

Doctor:

- Can search availability.
- Cannot access internal pharmacy data.
- Cannot access prescriptions in MVP.
- Cannot create prescriptions in MVP unless added later.

Public user:

- Can use public search.
- Can view public pharmacy contact details.
- Cannot see exact quantities.
- Cannot see prescriptions, invoices, costs, suppliers, or staff.

## 8.4 Protected Routes

Public:

- /
- /search
- /login

Pharmacy protected:

- /pharmacy/dashboard
- /pharmacy/inventory
- /pharmacy/inventory/:id
- /pharmacy/imports
- /pharmacy/sales
- /pharmacy/sales/new
- /pharmacy/invoices/:id

- /pharmacy/prescriptions
- /pharmacy/settings
- /pharmacy/staff

Admin protected:

- /admin
- /admin/pharmacies
- /admin/medicines
- /admin/imports
- /admin/audit-logs
- /admin/users

Doctor protected, optional:

- /doctor/search

---

# 9. Technical Requirements

## 9.1 Recommended Repository Structure

Use a monorepo:

```
medisync/
  apps/
    web/
      app/
      components/
      lib/
      hooks/
      types/
      public/
      package.json
    api/
      config/
      apps/
        accounts/
        pharmacies/
        medicines/
        inventory/
        imports/
        sales/
        prescriptions/
        audit/
      manage.py
```

```
    requirements.txt
  docs/
    PRD.md
    API.md
    SETUP.md
  docker-compose.yml
  .env.example
  README.md
```

## 9.2 Frontend Structure

Use Next.js with TypeScript.

Recommended frontend folders:

```
apps/web/
  app/
    page.tsx
    search/
      page.tsx
    login/
      page.tsx
    pharmacy/
      layout.tsx
      dashboard/
        page.tsx
      inventory/
        page.tsx
        [id]/
          page.tsx
        new/
          page.tsx
      imports/
        page.tsx
        [id]/
          page.tsx
      sales/
        page.tsx
        new/
          page.tsx
      invoices/
        [id]/
          page.tsx
      prescriptions/
        page.tsx
      settings/
```

```
        page.tsx
      staff/
        page.tsx
    admin/
      layout.tsx
      page.tsx
      pharmacies/
        page.tsx
      medicines/
        page.tsx
      users/
        page.tsx
      audit-logs/
        page.tsx
  components/
    ui/
    layout/
    forms/
    tables/
    medicine/
    inventory/
    sales/
  lib/
    api-client.ts
    auth.ts
    validators.ts
    constants.ts
  types/
    api.ts
```

Frontend requirements:

- Use TypeScript.
- Use a shared API client wrapper.
- Use reusable UI components.
- Handle loading, empty, and error states consistently.
- Do not hardcode API URLs outside config.
- Do not store sensitive prescription data in browser storage.
- Do not expose private file URLs directly.
- Use route guards or middleware for protected pages.

## 9.3 Backend Structure

Use Django and Django REST Framework.

Recommended backend apps:

```
apps/api/apps/
  accounts/
  pharmacies/
  medicines/
  inventory/
  imports/
  sales/
  prescriptions/
  audit/
```

Backend requirements:

- Use PostgreSQL.
- Use UUID primary keys.
- Use Django migrations.
- Use DRF serializers and viewsets or APIViews.
- Use service-layer functions for business logic.
- Use permissions classes for role checks.
- Use transactions for sales and stock deductions.
- Use audit logging for sensitive actions.
- Use private file storage for prescriptions.
- Keep business logic out of serializers where practical.
- Do not create microservices.

## 9.4 API Routes

Use REST-style API routes.

Authentication:

- POST /api/auth/login/
- POST /api/auth/logout/
- GET /api/auth/me/

Public:

- GET /api/public/search/?q=&area=&medicine_id=
- GET /api/public/pharmacies/:id/

Admin:

- GET /api/admin/pharmacies/
- POST /api/admin/pharmacies/
- GET /api/admin/pharmacies/:id/
- PATCH /api/admin/pharmacies/:id/
- GET /api/admin/users/

- POST /api/admin/users/
- PATCH /api/admin/users/:id/
- GET /api/admin/medicines/
- POST /api/admin/medicines/
- PATCH /api/admin/medicines/:id/
- GET /api/admin/audit-logs/
- GET /api/admin/imports/

Pharmacy:

- GET /api/pharmacy/dashboard/
- GET /api/pharmacy/profile/
- PATCH /api/pharmacy/profile/
- GET /api/pharmacy/inventory/
- POST /api/pharmacy/inventory/
- GET /api/pharmacy/inventory/:id/
- PATCH /api/pharmacy/inventory/:id/
- POST /api/pharmacy/inventory/:id/adjust/
- GET /api/pharmacy/stock-movements/
- GET /api/pharmacy/imports/
- POST /api/pharmacy/imports/upload/
- GET /api/pharmacy/imports/:id/
- POST /api/pharmacy/imports/:id/confirm/
- POST /api/pharmacy/imports/:id/cancel/
- GET /api/pharmacy/sales/
- POST /api/pharmacy/sales/
- GET /api/pharmacy/sales/:id/
- GET /api/pharmacy/invoices/:id/
- GET /api/pharmacy/prescriptions/
- POST /api/pharmacy/prescriptions/
- GET /api/pharmacy/prescriptions/:id/
- GET /api/pharmacy/prescriptions/:id/download/
- GET /api/pharmacy/staff/
- POST /api/pharmacy/staff/
- PATCH /api/pharmacy/staff/:id/

Medicine:

- GET /api/medicines/search/?q=
- GET /api/medicines/:id/

## 9.5 Database Requirements

- PostgreSQL required.
- Use database indexes on:
- User.email.
- Pharmacy.city.
- Pharmacy.area.

- Medicine.brand_name.
- Medicine.generic_name.
- MedicineAlias.alias.
- InventoryBatch.pharmacy_id.
- InventoryBatch.medicine_id.
- InventoryBatch.expiry_date.
- InventoryBatch.current_quantity.
- Sale.pharmacy_id.
- Sale.sale_datetime.
- AuditLog.pharmacy_id.
- AuditLog.created_at.
- Use transactions for:
- Sale creation.
- Stock deduction.
- Import confirmation.
- Sale cancellation, if implemented.
- Use soft-deactivation instead of hard deletion for:
- Users.
- Pharmacies.
- Medicine records.
- Inventory batches.

## 9.6 File Upload and Storage

Prescription files:

- Allowed types:
- PDF.
- JPG.
- JPEG.
- PNG.
- Maximum file size:
- Configurable using environment variable.
- Suggested MVP default: 10 MB.
- Storage:
- Local private storage in development.
- S3 private bucket in production.
- Access:
- Never expose public file URLs.
- Use backend-authenticated download route.
- Log download/view actions.

Import files:

- Allowed types:
- CSV.
- XLSX.

- Maximum file size:
- Configurable.
- Suggested MVP default: 5 MB.

## 9.7 Environment Variables

Create `.env.example` with:

```
# Backend
DJANGO_SECRET_KEY=
DJANGO_DEBUG=
DJANGO_ALLOWED_HOSTS=
DATABASE_URL=
CORS_ALLOWED_ORIGINS=
CSRF_TRUSTED_ORIGINS=

# Storage
USE_S3=
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=
AWS_STORAGE_BUCKET_NAME=
AWS_S3_REGION_NAME=
AWS_S3_ENDPOINT_URL=
MAX_PRESCRIPTION_FILE_SIZE_MB=
MAX_IMPORT_FILE_SIZE_MB=

# Frontend
NEXT_PUBLIC_API_BASE_URL=

# Security
SESSION_COOKIE_SECURE=
CSRF_COOKIE_SECURE=

# Optional
SENTRY_DSN=
```

## 9.8 Security Considerations

- Use HTTPS in production.
- Enforce backend authorization on every protected API.
- Never trust frontend role checks alone.
- Use private storage for prescriptions.
- Avoid storing unnecessary patient data.
- Do not expose exact stock quantity publicly.
- Audit prescription access.
- Audit inventory quantity changes.

- Prevent cross-pharmacy data access.
- Validate uploaded file type and size.
- Sanitize file names.
- Rate-limit public search if possible.
- Use secure cookies if using session authentication.
- Use CORS and CSRF settings carefully.
- Do not log sensitive prescription content.
- Use least-privilege AWS credentials.
- Use database backups in production.
- Add error handling that does not leak internal stack traces.

---

# 10. Design Requirements

## 10.1 UI Style

The interface should feel clean, trustworthy, and operational. Avoid playful medical visuals or overly futuristic AI branding. This is a pharmacy operations tool, not a sci-fi dashboard.

Style direction:

- Clean healthcare SaaS.
- Light background.
- Clear tables.
- Strong form usability.
- Calm colors.
- High readability.
- Minimal visual noise.

## 10.2 Layout Principles

- Use a dashboard layout for pharmacy and admin pages.
- Use sidebar navigation on desktop.
- Use top navigation or collapsible menu on mobile.
- Keep primary actions visible.
- Use tables for inventory, sales, imports, and audit logs.
- Use cards for dashboard metrics.
- Use badges for status indicators.
- Use confirmation dialogs for destructive or sensitive actions.

## 10.3 Components

Reusable components:

- Button.
- Input.

- Select.
- Textarea.
- Modal.
- Table.
- Badge.
- Alert.
- Toast.
- Card.
- Date picker.
- File upload dropzone.
- Search combobox.
- Pagination.
- Empty state.
- Loading skeleton.
- Error state.
- Confirmation dialog.

## 10.4 Status Indicators

Use clear text and icons, not only color.

Examples:

- Available.
- Low stock.
- Unavailable.
- Expiring soon.
- Expired.
- Import failed.
- Import confirmed.
- Match required.
- Active.
- Inactive.

## 10.5 Responsiveness

- Public search must work well on mobile.
- Pharmacy dashboard should be usable on tablets and laptops.
- Tables should support horizontal scrolling on small screens.
- Critical actions should remain accessible on mobile.

## 10.6 Accessibility

- All form fields must have labels.
- Buttons must have clear text.
- Use sufficient color contrast.
- Do not rely on color alone for meaning.

- Support keyboard navigation for forms.
- Error messages should be readable and specific.
- Use semantic HTML where possible.

## 10.7 Branding Notes

MVP branding can be minimal:

- Product name: MediSync.
- Tone: professional, safe, reliable.
- Avoid making medical claims.
- Add disclaimer on public search: “Availability information is provided by connected pharmacies and may change. Please confirm with the pharmacy before visiting or using any medication.”

---

# 11. MVP Scope

## 11.1 Must-Have for MVP

1. Login/logout.
2. Role-based access.
3. Platform admin pharmacy management.
4. Medicine catalog management.
5. Pharmacy dashboard.
6. Inventory batch management.
7. Stock movement tracking.
8. Low-stock alerts.
9. Expiry alerts.
10. CSV/XLSX import with preview.
11. Manual resolution for unmatched medicine names.
12. Public medication availability search.
13. Sales recording.
14. Basic invoice detail.
15. Prescription upload/storage.
16. Audit logs for sensitive actions.
17. Private file access.
18. Basic responsive UI.
19. README and setup documentation.

## 11.2 Nice-to-Have Later

1. Doctor accounts.
2. Doctor-specific dashboard.
3. Prescription QR codes.
4. Pharmacy verification badge.
5. Printable invoice PDF.

6. Advanced reporting.
7. Demand forecasting.
8. Anomaly detection.
9. Supplier order suggestions.
10. Patient wallet.
11. SMS or WhatsApp notifications.
12. Map-based pharmacy search.
13. Barcode scanning.
14. Multi-branch pharmacy chains.
15. Integration with pharmacy POS systems.
16. Multi-language support.
17. Medicine safety checker with professional validation.
18. Admin data-quality workflow for duplicate medicine records.

## 11.3 Out of Scope

1. Direct treatment advice.
2. AI diagnosis.
3. Full e-prescription network.
4. Insurance claims.
5. Payment processing.
6. Delivery management.
7. Controlled-medication regulatory automation.
8. National regulator dashboard.
9. Mobile apps.
10. Microservices.
11. Kubernetes.
12. Blockchain.
13. Complex third-party integrations.
14. Public patient profiles.
15. Public prescription lookup.
16. Automatic medicine substitution without pharmacist approval.

---

# 12. Implementation Plan for Claude Code

# Phase 1: Project Setup and Foundations

### What Claude Code Should Build

- Monorepo structure.
- Django backend project.
- Next.js frontend project.
- Docker Compose for local PostgreSQL.
- Basic environment variable setup.
- README with local setup instructions.

- Basic health check endpoint.

### Files/Folders Likely Involved

```
README.md
.env.example
docker-compose.yml
apps/api/
apps/api/config/
apps/api/manage.py
apps/api/requirements.txt
apps/web/
apps/web/package.json
apps/web/app/
```

### Important Constraints

- Do not add unnecessary dependencies.
- Use PostgreSQL locally through Docker.
- Keep settings split cleanly for development and production.
- Do not hardcode secrets.
- Use TypeScript on frontend.

### Manual Testing Checklist

- Backend server starts.
- Frontend server starts.
- PostgreSQL container starts.
- Backend connects to database.
- Health check endpoint returns success.
- README setup steps are accurate.

---

## Phase 2: Authentication, Users, Roles, and Permissions

### What Claude Code Should Build

- Custom user model or extended Django user profile.
- Role enum.
- Pharmacy relation for users.
- Login/logout/me endpoints.
- Frontend login page.
- Frontend auth state handling.
- Protected route behavior.
- Backend permission classes.

### Files/Folders Likely Involved

```
apps/api/apps/accounts/
apps/api/apps/accounts/models.py
apps/api/apps/accounts/serializers.py
apps/api/apps/accounts/views.py
apps/api/apps/accounts/permissions.py
apps/api/apps/accounts/urls.py
apps/web/app/login/page.tsx
apps/web/lib/auth.ts
apps/web/lib/api-client.ts
```

### Important Constraints

- Backend permissions are mandatory.
- Frontend guards are not enough.
- Do not allow cross-pharmacy access.
- Public search must remain unauthenticated.

### Manual Testing Checklist

- Valid user can log in.
- Invalid login shows error.
- /api/auth/me returns current user.
- Staff cannot access admin route.
- Admin can access admin route.
- Inactive user cannot log in.

---

## Phase 3: Pharmacy and Admin Management

### What Claude Code Should Build

- Pharmacy model.
- Admin pharmacy CRUD APIs.
- Admin pharmacy management frontend.
- Pharmacy profile endpoint.
- Pharmacy settings page.
- Pharmacy activation/deactivation behavior.

### Files/Folders Likely Involved

```
apps/api/apps/pharmacies/
apps/web/app/admin/pharmacies/page.tsx
apps/web/app/pharmacy/settings/page.tsx
```

```
apps/web/components/forms/PharmacyForm.tsx
apps/web/components/tables/PharmacyTable.tsx
```

### Important Constraints

- Only platform admins can create/deactivate pharmacies.
- Pharmacy owners can update only their own pharmacy profile.
- Deactivated pharmacies should be blocked from pharmacy dashboard.
- Public search excludes inactive/private pharmacies.

### Manual Testing Checklist

- Admin creates pharmacy.
- Admin deactivates pharmacy.
- Pharmacy owner edits own profile.
- Staff cannot edit restricted pharmacy fields.
- Inactive pharmacy no longer appears publicly.

---

## Phase 4: Medicine Catalog

### What Claude Code Should Build

- Medicine model.
- Medicine alias model.
- Admin medicine CRUD APIs.
- Medicine search endpoint.
- Admin medicine catalog frontend.
- Basic duplicate prevention.

### Files/Folders Likely Involved

```
apps/api/apps/medicines/
apps/web/app/admin/medicines/page.tsx
apps/web/components/medicine/MedicineSearch.tsx
apps/web/components/forms/MedicineForm.tsx
```

### Important Constraints

- Do not hard delete medicines used by inventory.
- Search by brand, generic, and alias.
- Do not allow AI-generated medical advice.
- Medicine notes must be non-clinical unless professionally validated.

### Manual Testing Checklist

- Admin creates medicine.
- Admin adds aliases.
- Search finds brand name.
- Search finds generic name.
- Search finds alias.
- Duplicate warning appears.

---

## Phase 5: Inventory and Stock Movements

### What Claude Code Should Build

- InventoryBatch model.
- StockMovement model.
- Inventory CRUD APIs.
- Stock adjustment endpoint.
- Inventory list page.
- Add/edit batch page.
- Batch detail page.
- Low-stock and expiry calculations.

### Files/Folders Likely Involved

```
apps/api/apps/inventory/
apps/web/app/pharmacy/inventory/page.tsx
apps/web/app/pharmacy/inventory/new/page.tsx
apps/web/app/pharmacy/inventory/[id]/page.tsx
apps/web/components/inventory/
```

### Important Constraints

- Quantity changes must create StockMovement.
- Do not silently overwrite stock quantity.
- Quantity cannot go negative.
- Inventory APIs must be scoped to user’s pharmacy.
- Expired stock is not publicly available.

### Manual Testing Checklist

- Staff adds inventory batch.
- Staff adjusts quantity.
- Stock movement is created.
- Low-stock badge appears.
- Expired batch is flagged.

- Staff cannot view other pharmacy inventory.

---

## Phase 6: CSV/XLSX Import

### What Claude Code Should Build

- InventoryImport model.
- InventoryImportRow model.
- File upload endpoint.
- Parser for CSV/XLSX.
- Import preview API.
- Basic medicine matching service.
- Confirm import endpoint.
- Import page frontend.
- Import summary page.

### Files/Folders Likely Involved

```
apps/api/apps/imports/
apps/api/apps/imports/services/parser.py
apps/api/apps/imports/services/matching.py
apps/web/app/pharmacy/imports/page.tsx
apps/web/app/pharmacy/imports/[id]/page.tsx
apps/web/components/imports/
```

### Important Constraints

- Do not write inventory records until confirmation.
- Do not invent medicine catalog records automatically.
- Unmatched rows require manual match or skip.
- Invalid rows should show clear messages.
- Confirm import must run in a database transaction.

### Manual Testing Checklist

- Upload valid CSV.
- Upload valid XLSX.
- Missing required column shows error.
- Invalid quantity shows row error.
- Matched rows preview correctly.
- Unmatched rows can be skipped.
- Confirm creates inventory and stock movements.

---

## Phase 7: Public Availability Search

### What Claude Code Should Build

- Public search API.
- Availability calculation logic.
- Public search frontend.
- Area filter.
- Result cards.
- Availability badges.
- Disclaimer.

### Files/Folders Likely Involved

```
apps/api/apps/inventory/services/availability.py
apps/api/apps/medicines/services/search.py
apps/web/app/search/page.tsx
apps/web/components/medicine/PublicMedicineSearch.tsx
apps/web/components/medicine/AvailabilityResults.tsx
```

### Important Constraints

- Do not expose exact quantities.
- Do not show inactive/private pharmacies.
- Do not count expired stock as available.
- Use simplified status only.
- Include last updated timestamp.
- Include confirmation disclaimer.

### Manual Testing Checklist

- Public user searches medicine.
- Results show public-safe fields only.
- Exact stock quantity is hidden.
- Expired stock does not appear available.
- Private pharmacy does not appear.
- Misspelled or partial search still returns reasonable matches.

---

## Phase 8: Sales and Invoices

### What Claude Code Should Build

- Sale model.
- SaleItem model.
- Sale creation API.

- FEFO stock deduction service.
- Invoice detail API.
- Sales list frontend.
- Create sale frontend.
- Invoice detail frontend.

### Files/Folders Likely Involved

```
apps/api/apps/sales/
apps/api/apps/sales/services/create_sale.py
apps/web/app/pharmacy/sales/page.tsx
apps/web/app/pharmacy/sales/new/page.tsx
apps/web/app/pharmacy/invoices/[id]/page.tsx
apps/web/components/sales/
```

### Important Constraints

- Sale creation must be transactional.
- Stock deduction must create StockMovement.
- Do not allow sale when stock is insufficient.
- Use earliest expiry first.
- Completed sales should not be directly edited.
- Cancellation can be omitted from MVP if time is limited.

### Manual Testing Checklist

- Staff creates sale.
- Stock decreases.
- Invoice total is correct.
- Insufficient stock blocks sale.
- Multiple line items work.
- Invoice detail loads.
- Other pharmacy cannot access invoice.

---

## Phase 9: Prescription Records

### What Claude Code Should Build

- PrescriptionRecord model.
- Prescription upload API.
- Private file download API.
- Prescription list page.
- Prescription upload UI.
- Link prescription to sale.
- Audit prescription view/download.

### Files/Folders Likely Involved

```
apps/api/apps/prescriptions/
apps/web/app/pharmacy/prescriptions/page.tsx
apps/web/components/prescriptions/
```

### Important Constraints

- Files must be private.
- Public URLs are forbidden.
- Backend must authorize downloads.
- Prescription access must be audit logged.
- Validate file type and size.
- Do not store unnecessary patient data.

### Manual Testing Checklist

- Staff uploads prescription.
- File type validation works.
- File size validation works.
- Authorized user can download file.
- Unauthorized user cannot download file.
- Public user cannot access file.
- Audit log records prescription download.

---

## Phase 10: Dashboard, Alerts, and Audit Logs

### What Claude Code Should Build

- Pharmacy dashboard API.
- Dashboard frontend metrics.
- Alert lists.
- AuditLog model/service.
- Admin audit logs page.
- Pharmacy audit logs page if time allows.

### Files/Folders Likely Involved

```
apps/api/apps/audit/
apps/api/apps/pharmacies/services/dashboard.py
apps/web/app/pharmacy/dashboard/page.tsx
apps/web/app/admin/audit-logs/page.tsx
apps/web/components/dashboard/
apps/web/components/audit/
```

### Important Constraints

- Dashboard queries must be scoped by pharmacy.
- Audit logs should be append-only from normal flows.
- Do not include sensitive prescription content in logs.
- Keep dashboard simple and fast.

### Manual Testing Checklist

- Dashboard shows low-stock count.
- Dashboard shows expiring-soon count.
- Dashboard shows sales today.
- New pharmacy dashboard shows empty state.
- Inventory adjustment creates audit log.
- Prescription download creates audit log.
- Admin can filter audit logs.

---

## Phase 11: Polish, Documentation, and Deployment Prep

### What Claude Code Should Build

- Consistent loading/error/empty states.
- Basic responsive improvements.
- README updates.
- API documentation.
- Seed data command.
- Production settings checklist.
- Basic tests for critical permissions and stock logic.

### Files/Folders Likely Involved

```
README.md
docs/API.md
docs/SETUP.md
apps/api/apps/*/tests/
apps/api/apps/*/management/commands/seed_demo.py
apps/web/components/ui/
```

### Important Constraints

- Do not over-polish before must-have functionality works.
- Tests should focus on permissions, stock deduction, imports, and prescription access.
- Keep documentation accurate.

### Manual Testing Checklist

- Full demo flow works from login to sale.
- Public search works.
- Import flow works.
- Prescription upload/download works.
- Admin can manage pharmacy and medicine catalog.
- Permission boundaries are tested.
- README setup works from scratch.

---

## 13. Claude Code Instructions

Claude Code, follow these instructions strictly:

1. Build only what is described in this PRD.
2. Do not invent features beyond the PRD.
3. When something is ambiguous, ask for clarification instead of guessing.
4. Prioritize MVP functionality before polish.
5. Use clean, maintainable code.
6. Keep components modular and reusable.
7. Avoid unnecessary dependencies.
8. Do not introduce microservices.
9. Do not introduce Kubernetes, Kafka, blockchain, or complex infrastructure.
10. Use Django and Django REST Framework for the backend.
11. Use Next.js and TypeScript for the frontend.
12. Use PostgreSQL as the database.
13. Keep business logic in service functions where practical.
14. Keep backend authorization checks mandatory.
15. Do not rely only on frontend route protection.
16. Scope all pharmacy data by the authenticated user’s pharmacy.
17. Never expose prescription files publicly.
18. Never expose exact stock quantities in public search.
19. Never generate medical advice, treatment advice, or diagnosis features.
20. Do not automatically create medicine catalog records from imports unless explicitly approved by an admin or user flow in the PRD.
21. Use database transactions for stock-changing operations.
22. Create audit logs for sensitive actions.
23. Write simple tests for permission boundaries and stock logic.
24. Update README and documentation when setup or API behavior changes.
25. Do not change unrelated files.
26. Keep commits and changes focused by phase.
27. Before adding a dependency, explain why existing tools are insufficient.
28. Prefer boring, reliable implementation over clever abstractions.
29. Make empty, loading, and error states clear.
30. Keep the UI professional, readable, and suitable for pharmacy operations.
31. Preserve this PRD as the source of truth during implementation.